Tuplas como Registros, Filtros, Tabelas de Frequência e Consultas com Tabelas de Referência

---

PARTE 1: FUNDAMENTOS

1. Introdução: Dados Estruturados com Tuplas

No mundo real, um registro pode conter tanto dados simples quanto coleções. Uma tupla pode armazenar qualquer tipo de dado, incluindo listas!

Infográfico 1: Estruturas de Dados

```
TUPLA SIMPLES (Registro Básico)
┌──────────────────────────────────────┐
│ ("Minecraft", "Sandbox", 200)        │
│     0:String    1:String   2:int     │
└──────────────────────────────────────┘

TUPLA COM LISTA (Registro Avançado)
┌──────────────────────────────────────────────────┐
│ ("Minecraft", "Sandbox", 200, [1.2, 2.4, 1.8])   │
│     0:String    1:String   2:int  3:Lista<float> │
└──────────────────────────────────────────────────┘

TUPLA DE TUPLAS (Coleção de Registros)
┌──────────────────────────────────────────────────┐
│ (                                                │
│   ("Valorant", "FPS", 120, [2.5, 3.0, 2.8]),     │
│   ("Minecraft", "Sandbox", 200, [5.1, 4.8])      │
│ )                                                │
└──────────────────────────────────────────────────┘
```

```python
# Uma tupla como um registro de um jogo
jogo_simples = ("Minecraft", "Sandbox", 200)

# Uma tupla que contém uma lista (ex: médias de horas por mês)
jogo_avancado = ("Minecraft", "Sandbox", 200, [1.2, 2.4, 1.8, 2.1])
```

2. Coleção de Registros: A Tupla de Tuplas

Para organizar vários registros, criamos uma coleção principal imutável: uma **tupla** que contém várias **tuplas-registro**. (tuplinhas)

```python
jogos = (
    ("Valorant", "FPS", 120, [2.5, 3.0, 2.8, 3.2]),      # Médias de horas por mês
    ("Minecraft", "Sandbox", 200, [5.1, 4.8, 6.0, 5.5]),
    ("FIFA", "Esporte", 80, [1.2, 1.5, 1.3, 1.4]),
    ("CS2", "FPS", 150, [3.5, 3.2, 3.8, 3.0]),
    ("GTA V", "Ação", 170, [4.0, 4.2, 3.9, 4.5]),
```

```
    ("Fortnite", "BattleRoyale", 90, [1.8, 2.0, 1.9, 2.2]),
    ("Rocket League", "Esporte", 60, [0.8, 1.0, 1.1, 0.9]),
    ("The Witcher 3", "RPG", 45, [0.5, 0.6, 0.7, 0.4]),
    ("Call of Duty", "FPS", 110, [2.2, 2.5, 2.3, 2.4])
)
```

3. Percorrendo com Desempacotamento e Listas Internas

O desempacotamento funciona mesmo quando a tupla contém listas.

Infográfico 2: Fluxo de Desempacotamento

```
TUPLA PRINCIPAL
│
├── Registro 0: ("Valorant", "FPS", 120, [2.5, 3.0, 2.8, 3.2])
│                │           │       │     └─► m1  m2  m3  m4
│                nome     gênero   horas   médias_mensais
│
└── for (nome, genero, horas, medias_mensais) in jogos:
                          │
                          ▼
               Podemos acessar a lista interna!
               media_anual = sum(medias_mensais) / len(medias_mensais)
               maior_media = max(medias_mensais)
```

```python
print("--- Lista de Jogos com Médias Mensais ---")
print(f"{'JOGO':<15} {'GÊNERO':<12} {'HORAS':>6} {'MÉDIA MENSAL':>15}")
print("-" * 55)

for (nome, genero, horas, medias) in jogos:
    media_anual = sum(medias) / len(medias)
    print(f"{nome:<15} {genero:<12} {horas:>6} {media_anual:>14.2f}h")
```

4. Aplicando Filtros com Listas Internas

Os filtros podem utilizar tanto os dados simples quanto os dados da lista interna.

4.1. Filtro 1: Jogos de FPS

python
```python
print("\n--- Jogos com média de horas > 3h ---")
for (nome, genero, horas, medias) in jogos:
    if genero == "FPS":
        print(f"  {nome}: horas {horas})")
```

4.2. Filtro 2: Jogos com média mensal > 3

python
```python
print("\n--- Jogos com média mensal > 3  ---")
for (nome, genero, horas, medias) in jogos:
    media = sum(medias) / len(medias)
    if media > 3:
        print(f"  {nome}: média {media:.2f} (total: {horas}h)")
```

4.3. Filtro 3: Jogos de FPS com tendência de crescimento

python
```python
print("\n--- Jogos de FPS com tendência de crescimento ---")
for (nome, genero, horas, medias) in jogos:
    if genero == "FPS" and medias[-1] > medias[0]:
        crescimento = ((medias[-1] - medias[0]) / medias[0]) * 100
        print(f"{nome}: {medias[0]:.1f} → {medias[-1]:.1f} ({crescimento:.0f}% de crescimento)")
```

PARTE 2: FUNÇÃO DE BUSCA GENÉRICA (A ÚNICA FUNÇÃO QUE PRECISAMOS)

5. A Função `enumerate`

A função `enumerate` é extremamente útil quando precisamos tanto do índice quanto do valor durante uma iteração.

```
:
lista = ["FPS", "RPG", "Esporte"]
│
└── enumerate(lista) produz:
│   (0, "FPS"), (1, "RPG"), (2, "Esporte")
│
│
└── enumerate(lista, start=1) produz:
    (1, "FPS"), (2, "RPG"), (3, "Esporte")
```

6. FUNÇÃO DE BUSCA GENÉRICA (ÚNICA PARA TODOS OS EXEMPLOS)

REGRAS FUNDAMENTAIS:

1. **Toda busca deve usar esta única função**
2. **A função usa `enumerate`**
3. **Retorna `None` quando não encontra (padrão Pythonico)**
4. **Retorna o `índice` quando encontra**

```python
# ==========================================
# FUNÇÃO DE BUSCA GENÉRICA (USAREMOS EM TODOS OS EXEMPLOS)
# ==========================================

def buscar_por_chave(tabela, chave_busca, indice_chave=0):
    """
    FUNÇÃO DE BUSCA GENÉRICA
    Busca uma chave em uma tabela usando enumerate.

    Args:
        tabela: lista de listas/tuplas onde cada elemento tem uma chave
        chave_busca: valor a ser buscado
        indice_chave: posição da chave em cada elemento (0 = primeira posição)

    Returns:
```

```
        int: índice da chave se encontrada
        None: se a chave não for encontrada
    """
    for (indice, elemento) in enumerate(tabela):
        if elemento[indice_chave] == chave_busca:
            return indice
    return None  # Não encontrou - retorna None (padrão Pythonico)
```

PARTE 3: PADRÃO ÚNICO DE AGRUPAMENTO (TABELAS DE FREQUÊNCIA)

7. O Padrão Fundamental: "Buscar e Agrupar"

Tanto tabelas de frequência quanto agrupamentos seguem o MESMO padrão:

1. **BUSCAR** se a chave já existe na tabela (usando `buscar_por_chave`)
2. **Se NÃO existir** (`pos is None`): cria nova entrada com a chave e um valor inicial
3. **Se existir**: atualiza a entrada existente

Infográfico 3: Padrão

```text
DADOS ORIGINAIS                    TABELA DE FREQUÊNCIA (Lista Mutável)

┌──────────────────┐
│ item1: chave=A   │  ──────────────►  ┌──────────────────────────────┐
│ item2: chave=B   │                   │                              │
│ item3: chave=A   │                   │  ┌────────────────────────┐  │
│ item4: chave=C   │                   │  │ [chave_A, valor_acumulado] │  │
│ item5: chave=B   │                   │  │ [chave_B, valor_acumulado] │  │
└──────────────────┘                   │  │ [chave_C, valor_acumulado] │  │
                                       │  └────────────────────────┘  │
                                       └──────────────────────────────┘
                                                  │
                                                  ▼ CONVERTER

                                       ┌──────────────────────────────┐
                                       │ RESULTADO FINAL (Tupla)      │
                                       │ (                            │
                                       │   (chave_A, valor_final),    │
                                       │   (chave_B, valor_final),    │
                                       │   (chave_C, valor_final)     │
                                       │ )                            │
                                       └──────────────────────────────┘
```

8. Tipo 1: Tabela de Frequência (Contagem)

Agrupa por chave e conta ocorrências.

```python
print("\n" + "="*65)
print("TIPO 1: TABELA DE FREQUÊNCIA (CONTAGEM)")
print("="*65)

# Lista mutável: cada elemento será [gênero, contador]
```

```python
tabela_freq = []

for (nome, genero, horas, _ )in jogos:
    # PASSO 1: BUSCAR (usando a função genérica)
    posicao = buscar_por_chave(tabela_freq, genero, indice_chave=0)

    # PASSO 2: ATUALIZAR (baseado no resultado da busca)
    if posicao is None:
        # Gênero NÃO encontrado: adiciona um novo registro com contador 1
        tabela_freq.append([genero, 1])
        print(f"  [+] Novo gênero '{genero}' adicionado com contador 1.")
    else:
        # Gênero encontrado: incrementa o contador
        tabela_freq[posicao][1] += 1
        print(f"  [~] Gênero '{genero}' incrementado. Novo total: {tabela_freq[posicao][1]}")

# Converte para tupla imutável
for (i, item) in enumerate(tabela_freq):
    tabela_freq[i] = tuple(item)
tabela_freq = tuple(tabela_freq)

print("\n--- RESULTADO FINAL (Tupla Imutável) ---")
print(f"{'GÊNERO':<15} | {'FREQUÊNCIA':>5}")
print("-" * 25)
for genero, freq in tabela_freq:
    print(f"{genero:<15} | {freq:>5}")
```

9. Tipo 2: Tabela de Agrupamento (Lista de Elementos)

Agrupa por chave e armazena todos os elementos em uma lista.

```python
print("\n" + "="*65)
print("TIPO 2: TABELA DE AGRUPAMENTO (LISTA DE ELEMENTOS)")
print("="*65)

# Lista mutável: cada elemento será [gênero, [lista_de_jogos]]
tabela_grupos = []

for (nome, genero, horas, _) in jogos:
    # PASSO 1: BUSCAR (usando a função genérica)
    posicao = buscar_por_chave(tabela_grupos, genero, indice_chave=0)

    # PASSO 2: ATUALIZAR (baseado no resultado da busca)
    if posicao is None:
        # Gênero NÃO encontrado: cria nova entrada com lista contendo o jogo
        tabela_grupos.append([genero, [nome]])          # Poderia ser (gênero,[nome])
        print(f"  [+] Novo gênero '{genero}' criado. Adicionado '{nome}'.")
    else:
        # Gênero encontrado: adiciona o jogo à lista existente
        tabela_grupos[posicao][1].append(nome)
        print(f"  [~] Gênero '{genero}' atualizado. Adicionado '{nome}' à lista.")

# Converte para tupla imutável (as listas internas também viram tuplas)
for i, item in enumerate(tabela_grupos):
    (genero, jogos_lista) = item
    tabela_grupos[i] = (genero, tuple(jogos_lista))
tabela_grupos = tuple(tabela_grupos)

print("\n--- RESULTADO FINAL (Tupla Imutável) ---")
print(f"{'GÊNERO':<15} | {'QUANTIDADE':>10} | {'JOGOS'}")
print("-" * 60)
for genero, jogos_tuple in tabela_grupos:
```

```python
    jogos_str = ", ".join(jogos_tuple)
    print(f"{genero:<15} | {len(jogos_tuple):>10} | {jogos_str}")
```

10. Tipo 3: Tabela de Agrupamento com Múltiplos Valores

Agrupa por chave e armazena múltiplos tipos de dados em listas separadas.

```python
print("\n" + "="*65)
print("TIPO 3: TABELA DE AGRUPAMENTO (MÚLTIPLOS VALORES)")
print("="*65)

# Lista mutável: cada elemento será [gênero, [jogos], [horas], [médias]]
tabela_completa = []

for (nome, genero, horas, medias) in jogos:
    # PASSO 1: BUSCAR (usando a função genérica)
    posicao = buscar_por_chave(tabela_completa, genero, indice_chave=0)

    # PASSO 2: ATUALIZAR (baseado no resultado da busca)
    if posicao is None:
        # Gênero NÃO encontrado: cria nova entrada com todas as listas
        tabela_completa.append([genero, [nome], [horas], [medias]])
        print(f"  [+] Novo gênero '{genero}' criado.")
    else:
        # Gênero encontrado: adiciona dados às listas existentes
        tabela_completa[posicao][1].append(nome)
        tabela_completa[posicao][2].append(horas)
        tabela_completa[posicao][3].append(medias)
        print(f"  [~] Gênero '{genero}' atualizado.")

# Calcula estatísticas
print("\n--- ESTATÍSTICAS POR GÊNERO ---")
print(f"{'GÊNERO':<15} {'QTD':>5} {'TOTAL HORAS':>15} {'MÉDIA HORAS':>15} {'MÉDIA
MENSAL':>15}")
print("-" * 70)

resultados = []
for genero, jogos_list, horas_list, medias_list in tabela_completa:
    total_horas = sum(horas_list)
    media_horas = total_horas / len(horas_list)

    # Calcula média das médias mensais
    todas_medias = []
    for medias in medias_list:
        todas_medias.extend(medias)
    media_mensal = sum(todas_medias) / len(todas_medias)

    resultados.append((genero, len(jogos_list), total_horas, media_horas, media_mensal))
    print(f"{genero:<15} {len(jogos_list):>5} {total_horas:>15} {media_horas:>14.1f}h
{media_mensal:>14.2f}h")

# Converte para tupla final
resultados_final = tuple(resultados)
print("\n--- RESULTADO FINAL (Tupla de Tuplas) ---")
for item in resultados_final:
    print(f"  {item}")
```

11. Tipo 4: Grupos Pré-definidos (Sem Busca)

Quando os grupos são conhecidos antecipadamente, não precisamos de busca. Acessamos diretamente pelos índices.

```python
print("\n" + "="*65)
print("TIPO 4: GRUPOS PRÉ-DEFINIDOS")
print("="*65)

# Grupos pré-definidos como tupla (imutável)
grupos_base = (
    ("Pouco ( < 80h)", []),
    ("Médio (80-150h)", []),
    ("Muito (> 150h)", [])
)


print("\n--- Classificando Jogos por Grupo ---")
for nome, genero, horas, _ in jogos:
    if horas < 80:
        grupos_base [0][1].append(nome)
        print(f"  [+] '{nome}' ({horas}h) -> Grupo: Pouco")
    elif horas <= 150:
        grupos_base [1][1].append(nome)
        print(f"  [+] '{nome}' ({horas}h) -> Grupo: Médio")
    else:
        grupos_base [2][1].append(nome)
        print(f"  [+] '{nome}' ({horas}h) -> Grupo: Muito")

# Converte para tupla final
tabela_grupos=[]
for (i, item) in enumerate(grupos_base):
    (grupo, jogos_lista) = item
    tabela_grupos.append (grupo, tuple(jogos_lista))
tabela_grupos = tuple(tabela_grupos)

print("\n--- RESULTADO FINAL ---")
print(f"{'GRUPO':<20} | {'QUANTIDADE':>10} | {'JOGOS'}")
print("-" * 70)
for grupo, jogos_tuple in tabela_grupos:
    jogos_str = ", ".join(jogos_tuple) if jogos_tuple else "Nenhum jogo"
    print(f"{grupo:<20} | {len(jogos_tuple):>10} | {jogos_str}")
```

PARTE 4: COMBINAÇÃO DE TUPLAS

12. Combinando Tuplas com `zip`

Quando as tuplas estão na mesma ordem, podemos combiná-las facilmente.

```python
# Tupla de animais com suas características
tanimais = (
    ("Leão", "Carnívoro"),
    ("Elefante", "Herbívoro"),
    ("Tubarão", "Carnívoro"),
    ("Pinguim", "Carnívoro"),
    ("Girafa", "Herbívoro"),
)
```

```python
# Tupla de locais onde os animais vivem (mesma ordem)
tlocais = (
    ("Leão", "Savana"),
    ("Elefante", "Floresta"),
    ("Tubarão", "Oceano"),
    ("Pinguim", "Antártida"),
    ("Girafa", "Savana"),
)

def combinar_animais_locais(animais, locais):
    """Combina duas tuplas de informações sobre animais."""
    resultado = []
    for (nome, caracteristica), (_, habitat) in zip(animais, locais):
        resultado.append((nome, caracteristica, habitat))
    return tuple(resultado)

def exibir_animais_com_locais(animais_completos):
    """Exibe os animais com suas características e habitats."""
    print("\n--- ANIMAIS E SEUS HABITATS ---")
    for nome, caracteristica, habitat in animais_completos:
        print(f"  {nome} ({caracteristica}) vive em {habitat}")

# Testando
animais_completos = combinar_animais_locais(tanimais, tlocais)
exibir_animais_com_locais(animais_completos)

# Também podemos agrupar por característica usando nosso padrão
print("\n--- ANIMAIS AGRUPADOS POR CARACTERÍSTICA ---")
grupos_animais = []  # [caracteristica, [nomes]]

for (nome, característica) in tanimais:
    pos = buscar_por_chave(grupos_animais, caracteristica)
    if pos is None:
        grupos_animais.append([caracteristica, [nome]])
    else:
        grupos_animais[pos][1].append(nome)

for caracteristica, animais in grupos_animais:
    print(f"  {caracteristica}: {', '.join(animais)}")
```

13. Chaves Compostas

Quando a chave é formada por mais de um valor, usamos uma tupla como chave.

python
```python
compras = (
    ("João", "Livro", 1),
    ("Maria", "Caneta", 2),
    ("João", "Livro", 1),
    ("Maria", "Livro", 1),
    ("João", "Caderno", 3),
    ("Maria", "Livro", 2),
    ("Pedro", "Livro", 1),
    ("Pedro", "Caderno", 4),
    ("João", "Caderno", 1),
)

def agrupar_compras(compras):
    """
    Agrupa compras por cliente e produto, somando as quantidades.
    Usa chave composta (cliente, produto) como tupla.
    """
```

```python
    agrupado = []  # [[(cliente, produto), quantidade_total]]

    for cliente, produto, quantidade in compras:
        chave = (cliente, produto)

        # A função buscar_por_chave funciona com tuplas também!
        pos = buscar_por_chave(agrupado, chave, indice_chave=0)

        if pos is None:
            agrupado.append([chave, quantidade])
        else:
            agrupado[pos][1] += quantidade

    # Converte para tupla final
    return tuple((chave, total) for chave, total in agrupado)

# Testando
print("\n" + "="*70)
print("AGRUPAMENTO COM CHAVE COMPOSTA")
print("="*70)
resultado = agrupar_compras(compras)
for (cliente, produto), total in resultado:
    print(f"  (('{cliente}', '{produto}'), {total})")
```

PARTE 5: CONSULTAS COM TABELAS DE REFERÊNCIA

14. O Padrão de Consulta

Um padrão muito comum é ter **dados principais** e **tabelas de referência** (critérios, limites). Para cada dado principal, consultamos a tabela de referência usando a mesma função `buscar_por_chave`.

Infográfico 4: Padrão de Consulta

```text
DADOS PRINCIPAIS            TABELA DE REFERÊNCIA
┌─────────────────┐         ┌─────────────────┐
│ id: A, valor: X │         │ id: A, ref: R1  │
│ id: B, valor: Y │         │ id: B, ref: R2  │
│ id: C, valor: Z │         │ id: C, ref: R3  │
└─────────────────┘         └─────────────────┘
         │                           │
         └───────────┬───────────────┘
                     ▼
    PARA CADA DADO PRINCIPAL:
        pos = buscar_por_chave(tabela_ref, id)
        if pos is not None:
            referencia = tabela_ref[pos][1]
            # tomar decisão baseada na comparação
```

15. Exemplo 1: Curso de Idiomas

```python
# Dados principais: alunos
alunos_idiomas = (
```

```python
    ("Pedro", "Básico", 7.2),
    ("Carla", "Intermediário", 6.8),
    ("Thiago", "Avançado", 8.5),
    ("Julia", "Básico", 5.5),
    ("Mariana", "Intermediário", 8.0)
)

# Tabela de referência: critérios por nível
criterios_idiomas = (
    ("Básico", 6.0),
    ("Intermediário", 7.0),
    ("Avançado", 8.0)
)

def verificar_aprovacao(alunos, criterios):
    """Verifica aprovação comparando nota com critério do nível."""
    resultado = []

    for nome, nivel, nota in alunos:
        # Consulta a tabela de referência
        pos = buscar_por_chave(criterios, nivel, indice_chave=0)

        if pos is not None:
            nota_minima = criterios[pos][1]
            if nota >= nota_minima:
                situacao = "Aprovado"
            else
                situacao = "Reprovado"
        else:
            situacao = "Nível não encontrado"

        resultado.append(((nome, nivel, nota), situacao))

    return tuple(resultado)

# Testando
print("\n" + "="*70)
print("CONSULTA 1: CURSO DE IDIOMAS")
print("="*70)
resultado = verificar_aprovacao(alunos_idiomas, criterios_idiomas)
for (nome, nivel, nota), situacao in resultado:
    print(f"  {nome} ({nivel}): nota {nota} - {situacao}")
```

16. Exemplo 2: Controle de Estoque

python

```python
# Dados principais: produtos
produtos = (
    ("Arroz", "Alimentos", 10),
    ("Sabão", "Limpeza", 3),
    ("Detergente", "Limpeza", 5),
    ("Feijão", "Alimentos", 15),
    ("Leite", "Alimentos", 8)
)

# Tabela de referência: estoque mínimo por categoria
estoque_minimo = (
    ("Alimentos", 12),
    ("Limpeza", 4)
)

def verificar_estoque(produtos, minimos):
    """Identifica produtos com estoque abaixo do mínimo."""
    resultado = []
```

```python
    for (nome, categoria, quantidade) in produtos:
        # Consulta a tabela de referência
        pos = buscar_por_chave(minimos, categoria, indice_chave=0)

        if pos is not None:
            minimo = minimos[pos][1]
            if quantidade < minimo:
                comprar = minimo - quantidade
                resultado.append((nome, comprar))
                print(f"  [!] {nome}: estoque {quantidade} < mínimo {minimo}. Comprar
{comprar}.")
            else:
                print(f"  [OK] {nome}: estoque {quantidade} >= mínimo {minimo}")
        else:
            print(f"  [?] {nome}: categoria '{categoria}' sem mínimo definido")

    return tuple(resultado)

# Testando
print("\n" + "="*70)
print("CONSULTA 2: CONTROLE DE ESTOQUE")
print("="*70)
reposicao = verificar_estoque(produtos, estoque_minimo)
```

17. Exemplo 3: Monitoramento de Sensores

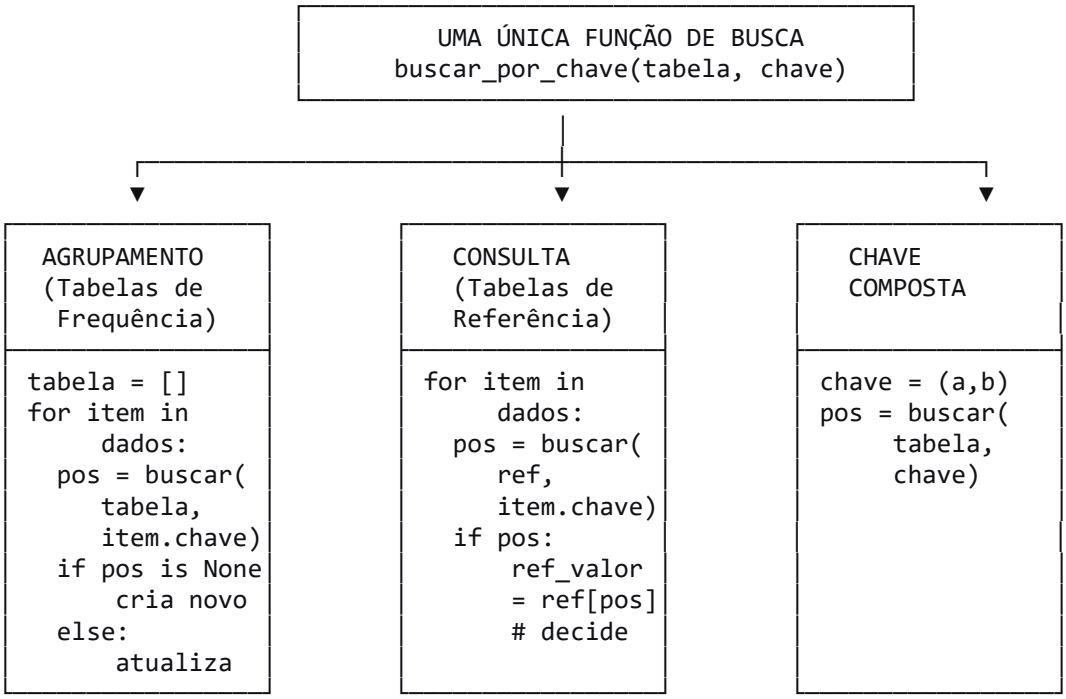
```python
# Dados principais: sensores
sensores = (
    ("Sala Máquinas", 75),
    ("Depósito", 50),
    ("Escritório", 22),
    ("Sala Máquinas", 80),
    ("Depósito", 58)
)

# Tabela de referência: limites por local
limites = (
    ("Sala Máquinas", 70),
    ("Depósito", 55),
    ("Escritório", 25)
)

def monitorar_sensores(sensores, limites):
    """Verifica se cada sensor está dentro do limite."""
    resultado = []

    for local, temperatura in sensores:
        # Consulta a tabela de referência
        pos = buscar_por_chave(limites, local, indice_chave=0)

        if pos is not None:
            maximo = limites[pos][1]
            status = "ALERTA" if temperatura > maximo else "OK"
            resultado.append((local, temperatura, maximo, status))
        else:
            resultado.append((local, temperatura, None, "Sem limite"))

    return tuple(resultado)

# Testando
print("\n" + "="*70)
print("CONSULTA 3: MONITORAMENTO DE SENSORES")
```

```python
print("="*70)
resultado = monitorar_sensores(sensores, limites)
for (local, temp, maximo, status) in resultado:
    if maximo:
        print(f"  {local}: {temp}°C (limite: {maximo}°C) - {status}")
    else:
        print(f"  {local}: {temp}°C - {status}")
```

PARTE 6: CONCEITOS FUNDAMENTAIS - RESUMO

Infográfico 5: Mapa Mental da Aula

text



REGRAS DE OURO:

Regra	Exemplo
1. Use a mesma função de busca	`pos = buscar_por_chave(tabela, chave)`
2. Teste com `is None`	`if pos is None:` ou `if pos is not None:`
3. Chave composta = tupla	`chave = (valor1, valor2)`
4. Converta listas para tuplas no final	`resultado = tuple(tuple(item) for item em tabela)`

Os 4 Tipos de Operação:

Tipo	Descrição	Quando usar
1. Contagem	`[chave, contador]`	Quando só precisa do total
2. Lista simples	`[chave, [elementos]]`	Quando precisa dos elementos
3. Listas múltiplas	`[chave, [e1], [e2], ...]`	Quando precisa de múltiplos atributos
4. Grupos pré-definidos	Acesso direto por índice	Quando as categorias são conhecidas

Padrões de Uso da Função Genérica:

```python
# Busca simples (chave na posição 0)
pos = buscar_por_chave(tabela, valor_busca)

# Busca com chave em outra posição
pos = buscar_por_chave(tabela, valor_busca, indice_chave=2)

# Busca com chave composta
chave = (valor1, valor2)
pos = buscar_por_chave(tabela, chave)
```